

PEC 3

Jornadas de Artesania

Reescritura con utility-first CSS y Tailwind v4 sobre UOC Boilerplate

Autor: Federico Javier Martino **Asignatura:** Herramientas HTML y CSS II **Actividad:** PEC 3 (25% de la nota de evaluacion continua) **Universidad:** Universitat Oberta de Catalunya (UOC) **Fecha de entrega:** 4 de junio de 2026

Repositorio GitHub	https://github.com/Federicojaviermartino/jornadas-artesania-pec3
Sitio publicado (Netlify)	https://jornadas-artesania-pec3.netlify.app/
Despliegue PEC 2 (referencia)	https://jornadas-artesania-uoc.netlify.app/
Video explicativo	<i>pendiente: enlace de Google Drive (@uoc.edu)</i>

Indice

1. Introduccion y objetivos
2. Entorno y proceso de desarrollo
3. Construcccion de las paginas
4. Clases y componentes extraidos
5. Preguntas sobre utility-first CSS
6. Pagina generada con una IA generativa
7. Enlaces
8. Conclusiones

1. Introduccion y objetivos

Esta practica revisa el sitio **Jornadas de Artesania** desarrollado en la PEC 2 y lo reescribe aplicando la metodologia **utility-first CSS** vista en el modulo 4.2, usando la libreria de utilidades **Tailwind CSS v4**. No se trata de mejorar el diseno ni el contenido, sino de reescribir el HTML y el CSS de dos paginas para que sigan el paradigma de Atomic CSS, partiendo de un proyecto nuevo basado en UOC Boilerplate (empaquetador Parcel, estilos en Sass) y sin Bootstrap ni Stylelint.

Concretamente, en esta entrega:

- Se parte de un proyecto limpio basado en UOC Boilerplate y se instala y configura Tailwind v4 con el enfoque CSS-first.
- Se recrean dos paginas de la PEC 2 (**Portada** y **Ponentes**) con utilidades de Tailwind, manteniendolas responsive.
- Se extraen clases con `@apply` y componentes con `posthtml-include`.
- Se anade una tercera pagina (**Concurso**) generada con ayuda de una IA generativa a partir de un diseno de Figma.
- Se documenta el proceso y se publica el repositorio y el despliegue.

2. Entorno y proceso de desarrollo

2.1. Punto de partida y limpieza del proyecto

El proyecto parte de UOC Boilerplate (Parcel + Sass), la misma base que la PEC 2, pero como proyecto y repositorio nuevos. El primer paso fue eliminar todo lo que no debe estar en esta entrega:

- Se desinstalo **Bootstrap** y `@popperjs/core` de `package.json` y se elimino su `@import` en los estilos y el `import` de su JavaScript en `main.js`.
- Se desinstalo **Stylelint** y sus plugins, se borro `.stylelintrc` y se quito el paso de Stylelint del build, que queda como `npm-run-all clean parcel:build`.
- Se mantuvieron `.npmrc` (con `legacy-peer-deps=true`) y `.posthtmlrc` (posthtml-include).

2.2. Instalacion de dependencias

Sobre el proyecto limpio se instalaron Tailwind v4 y su plugin de PostCSS, ademas de las fuentes e iconos:

```
npm install tailwindcss @tailwindcss/postcss --save-dev
npm install @fontsource/playfair-display @fontsource/source-sans-pro @fortawesome/fontawesome-free
```

2.3. Configuracion de Tailwind v4 (CSS-first)

Tailwind v4 abandona el fichero `tailwind.config.js` de la v3 y se configura desde el propio CSS. La integracion con Parcel + Sass se hizo en tres puntos:

a) Plugin de PostCSS en `.postcssrc`:

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

b) Importacion en `_dependencies.scss`, con extension `.css` para que Dart Sass lo deje pasar y lo resuelva `@tailwindcss/postcss`; las fuentes e iconos sin prefijo `npm:`:

```
@import "tailwindcss/index.css";
@import "@fortawesome/fontawesome-free/css/all.css";
@import "@fontsource/playfair-display/400.css";
```

c) Tema con `@theme` en `_variables.scss`, reutilizando paleta, tipografías, breakpoints, radios y sombras de la PEC 2 como tokens de Tailwind:

```
@theme {
  --font-serif: "Playfair Display", Georgia, serif;
  --text-hero: 4rem;
  --color-clay: #b85c2c;
  --color-ink: #2b2018;
  /* ... paleta, breakpoints sm/md/lg/xl, radios y sombras ... */
}
```

Así, utilidades como `text-clay`, `bg-cream` o `text-hero` generan los valores del tema. El orden de carga en `main.scss` es: variables (tema) → dependencias → base → componentes.

2.4. Entorno de desarrollo y compilación

Durante el desarrollo se usa `npm run dev` (Parcel en `localhost:8123` con recarga en caliente). La compilación se hace con `npm run build` (`clean` + `parcel:build`), que deja en `dist/` el HTML, el CSS minificado, el JS y los recursos. Tailwind solo emite las utilidades usadas y, por su *tree-shaking*, solo las variables de tema referenciadas, por lo que la hoja final es pequeña.

2.5. Publicación

El código se publica en un repositorio de GitHub nuevo y se despliega en Netlify (build `npm run build`, publish `dist`), distintos de los de la PEC 2. Las URLs figuran en el apartado 7.

3. Construcción de las páginas

3.1. Decisiones generales

Se eligieron **Portada** y **Ponentes** por ser las páginas con más estructuras repetidas (tarjetas de información, destacados, fichas de ponente y actividades), ideales para demostrar la extracción de clases y componentes. La portada se mantiene como `index.html`. La arquitectura de estilos quedó mínima y separada en capas de Tailwind:

- `_variables.scss`: tema `@theme` (y la paleta heredada como variables Sass de apoyo).
- `modules/_base.scss`: capa `base` con tipografía global, color de enlaces y enlace de salto accesible.
- `modules/_components.scss`: capa `components` con las clases extraídas vía `@apply`.

Las hojas de estilo por pagina de la PEC 2 (cientos de lineas de SCSS con BEM) desaparecen: el estilo vive ahora en las clases utilitarias del HTML.

3.2. Portada (index.html)

- **Hero / cartel:** el grid con `grid-template-areas` de la PEC 2 se rehizo con un grid responsive (`md:grid-cols-[1.3fr_1fr]`); el fondo (gradiente multicapa) se mantiene como clase `.hero-bg`.
- **Banda de informacion y destacados:** grids de 1 a 3 columnas con los componentes `info-item` y `highlight-card`; este ultimo conserva la **container query** (pasa a horizontal cuando su ancho supera 480px) con `@container` y `@[480px]:`.
- **Galeria:** el carrusel de Bootstrap, que requeria JavaScript, se representa de forma estatica (imagen, leyenda, indicadores y flechas), como permite el enunciado.

3.3. Ponentes (ponentes.html)

- **Cabecera interior:** banner con gradiente (`.banner-bg`), titulo, subtítulo y miga de pan.
- **Filtros:** botones `.btn-filter`; el filtrado lo gestiona el JavaScript propio (`main.js`), vanilla y sin dependencia de Bootstrap.
- **Fichas:** ocho instancias del componente `speaker-card` (incluida la keynote a ancho completo), con container query a 520px.
- **Programa:** el acordeon de Bootstrap se sustituye por el elemento nativo `<details>`, plegable sin JavaScript.

3.4. Cabecera, pie y problematicas

La navbar pasa a un `<nav>` flex: en movil se oculta el menu (`hidden lg:flex`) y se muestra la hamburguesa (`lg:hidden`), sin desplegarse. Las principales dificultades y su solucion:

Problema	Solucion
Dart Sass falla al importar <code>tailwindcss</code> (lo busca como parcial).	Importar <code>"tailwindcss/index.css"</code> : la extension <code>.css</code> hace que Sass lo deje pasar y Tailwind lo resuelva en PostCSS.
Resolucion de FontAwesome y Fontsource.	Importarlos sin prefijo <code>npm:</code> ; el build reescribe correctamente las rutas de las fuentes en <code>dist/</code> .
Usar <code>@apply</code> dentro de ficheros <code>.scss</code> .	Funciona porque todo compila a una unica hoja con el import de Tailwind; se evitan variantes con dos puntos dentro de <code>@apply</code> .
Componentes de Bootstrap con JS.	Carrusel estatico y acordeon con <code><details></code> nativo.
Tarjetas repetidas con datos distintos.	posthtml-include con <code>locals</code> (<code>{{ }}</code> , <code><if></code> y <code><each></code>).

4. Clases y componentes extraídos

La documentacion de Tailwind recomienda, ante la duplicacion, usar plantillas/partials para los bloques complejos y reservar `@apply` para patrones pequenos muy repetidos. Se aplican ambas estrategias.

4.1. Clases con @apply

Clase	Por que se extrae
<code>.btn</code> , <code>.btn-clay</code> , <code>.btn-cream-outline</code>	El mismo conjunto de utilidades de boton aparece en cabecera y hero.
<code>.btn-filter</code>	Los cinco filtros de ponentes comparten estilo; su estado <code>.active</code> (del JS) se define una vez.
<code>.pill</code>	Etiqueta redondeada (badges y tags); el color se pasa por utilidad.
<code>.card</code>	Tarjeta blanca con borde comun a destacados, ponentes y actividades.
<code>.section-heading</code>	Titular de seccion en tipografia serif.

4.2. Componentes con posthtml-include

Componente	Uso
<code>info-item.html</code>	Tarjeta de informacion practica (3 usos en la portada).
<code>highlight-card.html</code>	Tarjeta destacada con container query (6 usos en la portada).
<code>speaker-card.html</code>	Ficha de ponente parametrizada (8 usos), con bucle <code><each></code> para los enlaces.
<code>activity.html</code>	Actividad del programa (9 usos en ponentes).

Cada componente recibe sus datos con el atributo `locals`. La cabecera y el pie tambien son includes, pero no se cuentan entre estos tres minimos.

5. Preguntas sobre utility-first CSS

5.1. CSS semantico frente a CSS de utilidades

En las PEC anteriores se uso **CSS semantico** con BEM: clases con significado (`.speaker__name`) y el estilo en hojas SCSS aparte. Con el **CSS de utilidades** el estilo se compone en el HTML con clases atómicas (`flex`, `gap-2`, `text-clay`).

En el proceso de desarrollo: se deja de saltar entre HTML y SCSS (se maqueta y estiliza a la vez, mas rapido), desaparece el esfuerzo de nombrar clases, el tema centraliza las decisiones de diseno y casi desaparece el control de la especificidad.

En el codigo: el SCSS adelgaza drasticamente (cientos de lineas por pagina pasan a unas pocas clases `@apply` y la capa base); el HTML engorda (listas de clases largas), lo que se controla con componentes y `@apply`; y el CSS final es mas pequeno y predecible (solo se genera lo usado).

5.2. Libreria de componentes frente a libreria de utilidades

Bootstrap es una **libreria de componentes** (navbar, card, carousel ya disenados); Tailwind es una **libreria de utilidades** con la que se construye cualquier diseno.

	Componentes (Bootstrap)	Utilidades (Tailwind)
Arranque	Muy rapido: se pega el componente.	Mas lento: hay que construir cada pieza.
Personalizacion	Cuesta salirse del estilo de la libreria.	Total: diseno propio desde el inicio.
JavaScript	Carrusel/acordeon dependen de su JS.	No incluye JS; se resuelve aparte o con HTML nativo.
Peso final	Suele incluir CSS sin usar.	Solo se genera lo utilizado.

Se noto en los componentes con JS: el carrusel y el acordeon "venian gratis" en Bootstrap, mientras que con Tailwind hubo que decidir como resolverlos (carrusel estatico y `<details>`). A cambio, el resultado es a medida y sin dependencias de JavaScript de terceros.

6. Pagina generada con una IA generativa

6.1. Objetivo y diseno de partida

El enunciado pide usar una IA generativa para crear el HTML y el CSS de una pagina a partir de un diseno de Figma, afinando los prompts para acercarse al diseno y analizando el resultado. La pagina resultante (`concurso.html`) es autocontenida (HTML con su propio CSS, sin Tailwind, como permite el enunciado) y replica el diseno adaptando el contenido a la tematica artesana (un "Concurso de Artesania 2026").

6.2. Secuencia de prompts

Prompt 1. "A partir de esta imagen de Figma, genera una pagina HTML con su CSS (sin frameworks) que replique el diseno: barra superior con marca a la izquierda y menu con un boton oscuro a la derecha; titular centrado con subtítulo; imagen ancha; seccion con un título y tres tarjetas (título, parrafo y un autor con avatar); y un pie con el nombre, iconos sociales y tres columnas de enlaces. Contenedor centrado de unos 1080px."

Prompt 2. "Adapta el contenido a unas jornadas de artesanía: título 'Participantes', tres piezas (Cantaro vidriado, Cesto de mimbre, Bandeja de nogal) con autores artesanos, y el pie con columnas Piezas / Participantes / Ediciones (2026, 2025, 2024). Manten el aspecto neutro y claro del original."

Prompt 3. "Hazla responsive: tres tarjetas a una columna en moviles y el pie a dos y luego una columna. Reduce el titular en pantallas pequenas."

Prompt 4. "Los iconos sociales se ven rotos porque usabas clases de FontAwesome y la pagina no carga esa libreria. Sustituyelos por SVG en linea (facebook, linkedin, instagram, youtube) que hereden el color del texto."

6.3. Errores de la IA y correcciones

- **Iconos inexistentes:** uso clases `fa-*` sin cargar FontAwesome; se corrigio pidiendo SVG en linea (prompt 4).
- **Proporciones:** el primer resultado tenia menos aire que el diseno; se ajustaron margenes y `padding`.
- **Tipografia:** puso todo en sans-serif; el titular del diseno es serif, asi que se cambio a Georgia.

- **Contenido generico:** mantuvo textos del original ("recipes", "contestants") hasta pedirle adaptarlo a la tematica.

6.4. Corregir frente a escribir de cero. Opinion

Partir del código de la IA fue más rápido que escribir la página de cero: en pocos minutos había una estructura completa que solo necesitaba ajustes, fáciles de localizar porque el código era limpio y predecible. La dificultad de corregir crece cuando el código es enrevesado o usa técnicas que uno no domina; aquí no fue el caso. **Mi opinion** es que estas herramientas son muy útiles como punto de partida y para andamiaje, siempre que se entienda el código que producen y se revise, porque cometen errores (referencias a librerías no cargadas, accesibilidad, fidelidad al diseño) y tienden a soluciones genéricas. Son un acelerador, no un sustituto del criterio propio.

7. Enlaces

Recurso	URL
Repositorio GitHub (PEC 3)	https://github.com/Federicojaviermartino/jornadas-artesania-pec3
Despliegue Netlify (PEC 3)	https://jornadas-artesania-pec3.netlify.app/
Despliegue PEC 2 (referencia)	https://jornadas-artesania-uoc.netlify.app/
Video explicativo	<i>pendiente: enlace de Google Drive (@uoc.edu)</i>

8. Conclusiones

Reescribir el sitio con utility-first cambia por completo el flujo de trabajo: se maqueta y se estiliza a la vez, el CSS propio casi desaparece y la coherencia se mantiene gracias al tema. El reto se desplaza a controlar la duplicación en el HTML, que se resuelve con clases `@apply` y componentes de `posthtml-include`. Frente a una librería de componentes como Bootstrap, Tailwind exige más trabajo inicial pero da control total del diseño y un CSS final más ligero. La experiencia con la IA generativa confirma su utilidad como punto de partida, siempre con revisión crítica del código.